Implement Client Script and UI Policy: Enforcing Incident Data Integrity in ServiceNow

Project Description:

Implement Client Script and UI Policy is a ServiceNow application development project that demonstrates how client-side controls, UI Policies and Client Scripts, work together to enforce data integrity on Incident records. Field visibility, mandatory status, read-only behaviour, and save-time validation are controlled dynamically based on the value of the Impact field, so Incidents are created with complete and valid information.

The project centres on a single business rule: when an Incident is marked as High Impact, the platform automatically tightens the rules around related fields (Assignment Group, Urgency, Assigned To) and prevents the record from being saved until those rules are satisfied. It is delivered through the High Impact Control UI Policy with two UI Policy Actions and three Client Scripts (onChange, onSubmit, and onCellEdit) built with the g_form API on the Incident table, replacing reliance on manual checks that lead to incomplete or inconsistent data and affect triage, reporting accuracy, and SLA compliance.

Scenarios

Scenario 1:

An agent opens a new Incident and sets Impact to High. The High Impact Control UI Policy makes Assignment Group mandatory and Urgency read-only, and the onChange Client Script automatically sets Urgency to High and shows an information message. If Impact is later changed away from High, the mandatory and read-only rules reverse automatically.

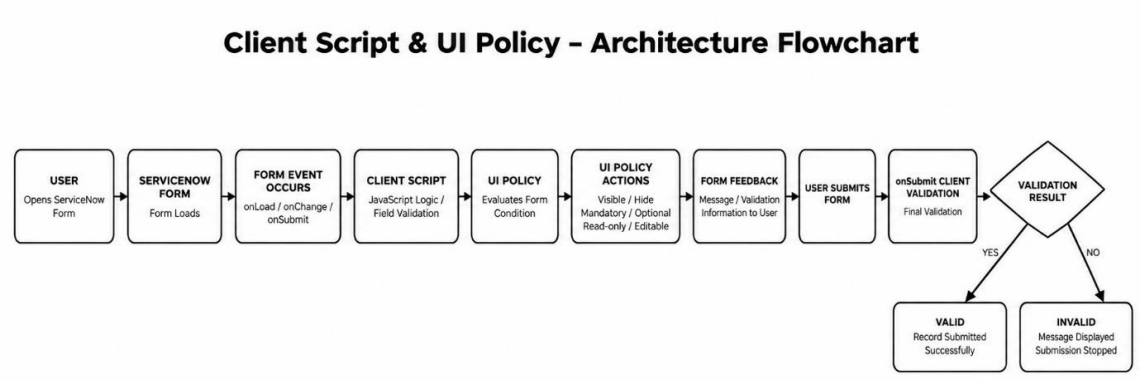
Scenario 2:

An agent sets Impact to High but leaves Assigned To empty and clicks Submit. The onSubmit Client Script blocks the save and displays the message “Assigned To is mandatory for High impact incidents” against the field. Once a user is selected in Assigned To, the Incident saves successfully.

Scenario 3:

An agent double-clicks the State field of an Incident in the list view to edit it directly. The onCellEdit Client Script shows an alert and cancels the edit, so State changes are made from the Incident form where all UI Policies and Client Scripts apply.

Technical Architecture:



Description:

The project follows a client-side ServiceNow form architecture built around the Incident table. The user opens the Incident form and form events (onChange on Impact, onSubmit, and onCellEdit on State in the list view) trigger the configured Client Scripts, while the High Impact Control UI Policy evaluates the condition Impact is 1 – High and applies its UI Policy Actions: Assignment group becomes mandatory and Urgency becomes read-only, both reversing when the condition is no longer met. The onChange script sets Urgency to High, the onSubmit script performs the final save-time validation of Assigned To, and the onCellEdit script blocks State edits from the list view. The result is either a valid record submitted successfully, or a message displayed and the submission stopped.

Data Model:

No custom database is used, so an ER diagram is not required. All logic runs on the out-of-the-box Incident table and the fields below.

Field (Label)	Backend Name	Role in This Project
Impact	impact	Controlling field; High Impact (value 1) triggers all downstream rules.
Urgency	urgency	Auto-set to High and made read-only when Impact is High.
Assignment group	assignment_group	Made mandatory by the High Impact Control UI Policy when Impact is High.
Assigned to	assigned_to	Checked by the onSubmit script; save is blocked if empty on a High Impact incident.
State	state	Protected from direct list-view edits by the onCellEdit script.

Pre-requisites:

- **ServiceNow Platform Knowledge:** ServiceNow Product Documentation
- **ServiceNow Instance with Admin Access:** ServiceNow Developer Program
- **UI Policy and Client Script Concepts:** ServiceNow Product Documentation
- **JavaScript Programming Proficiency:** MDN JavaScript Guide

- **g_form (GlideForm) API Familiarity:** ServiceNow Developer Site
- **Incident Management Basics:** ServiceNow Product Documentation
- **Web Browser:** A modern browser such as Chrome, Edge, or Firefox with a stable internet connection.

Project Workflow:

Milestone 1: Create UI Policy on Incident

- **Activity 1.1:** Create the High Impact Control UI Policy on the Incident table with the condition Impact is 1 – High.
- **Activity 1.2:** Add a UI Policy Action that makes Assignment group mandatory and enable Reverse if false.

Milestone 2: Create UI Policy Action for Urgency

- **Activity 2.1:** Add a UI Policy Action on the Urgency field and set it to Read-only while Impact is High.

Milestone 3: Create onChange Client Script

- **Activity 3.1:** Create the Auto set urgency for high impact Client Script (onChange on Impact).
- **Activity 3.2:** Write the script that sets Urgency to High and displays an information message.

Milestone 4: Create onSubmit Client Script (Save Validation)

- **Activity 4.1:** Create the Prevent save if Assigned To missing Client Script (onSubmit).
- **Activity 4.2:** Write the script that blocks the save when Assigned To is empty on a High Impact incident.

Milestone 5: Create onCellEdit Client Script

- **Activity 5.1:** Create the Prevent state change via list edit Client Script (onCellEdit on State).
- **Activity 5.2:** Write the script that shows an alert and cancels State edits made from the list view.

Milestone 6: Testing and Validation

- **Activity 6.1:** Test mandatory enforcement (save blocked when Assigned To is empty).
- **Activity 6.2:** Test successful save with Assigned To populated.
- **Activity 6.3:** Test the reverse condition when Impact changes away from High.
- **Activity 6.4:** Test list edit blocking on the Incident list view.

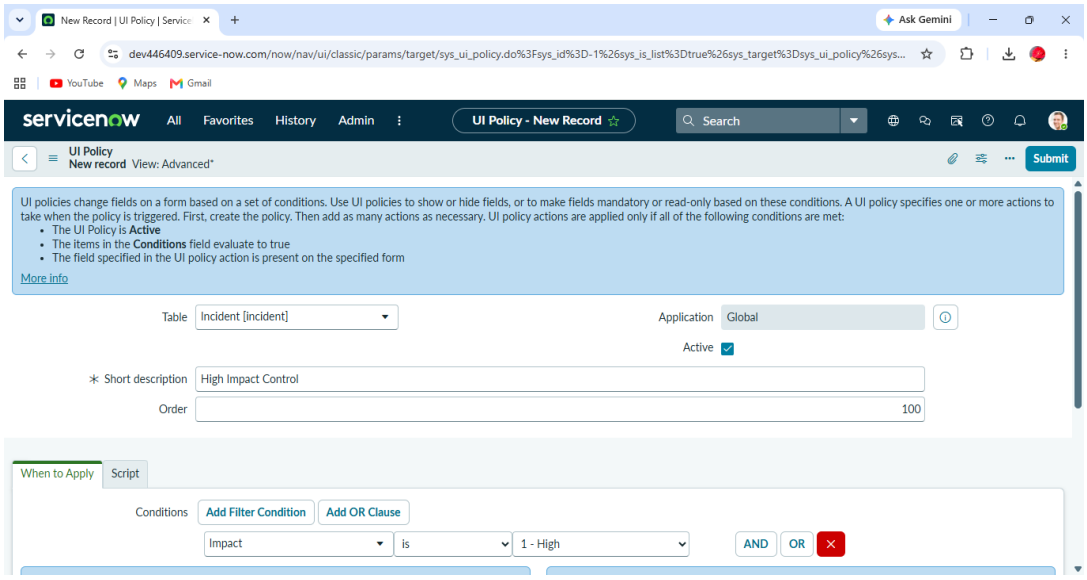
Milestone 7: Conclusion

Milestone 1: Create UI Policy on Incident

In this milestone, we create the High Impact Control UI Policy on the Incident table. High Impact incidents represent significant business disruption and must be routed to a team immediately, so making Assignment Group mandatory at the point of data entry removes the risk of a critical incident being saved without ownership.

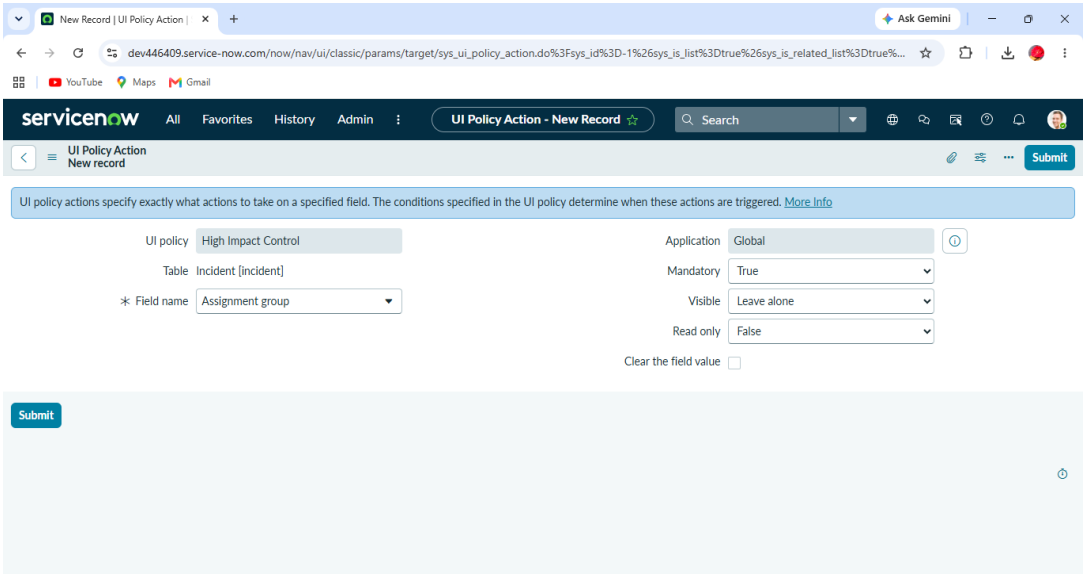
Activity 1.1: Create the UI Policy

- **Step 1 — Log in:** Log in to the ServiceNow instance with administrative or configuration access.
- **Step 2 — Open UI Policies:** Navigate to System UI → UI Policies and click New to create a new UI Policy record.
- **Step 3 — Enter the details:** Set the Short description to High Impact Control, the Table to Incident [incident], and keep Active checked.
- **Step 4 — Define the condition:** Under When to Apply, set Field = Impact, Operator = is, Value = 1 – High.



Activity 1.2: Add the UI Policy Action for Assignment Group

- **Step 1 — Add the action:** In the UI Policy Actions related list, click New and choose Assignment group as the Field name.
- **Step 2 — Set Mandatory:** Set Mandatory to True, leave Visible unchanged, and keep Read only as False.
- **Step 3 — Reverse if false:** Set Reverse if false to true so the change is undone automatically when Impact is no longer High.
- **Step 4 — Submit:** Review the configuration and click Submit to save.



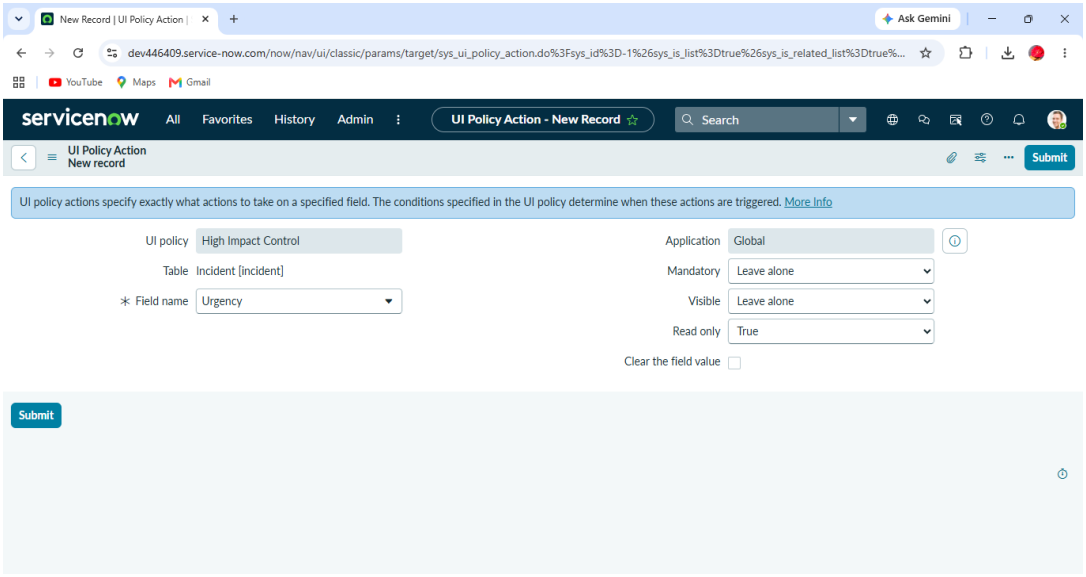
Field	Value
Name	High Impact Control
Table	Incident
Active	True
Condition	Impact is 1 – High
UI Policy Action Field	Assignment group
Mandatory	True
Read Only	False

Milestone 2: Create UI Policy Action for Urgency

Urgency should follow directly from Impact for High Impact incidents rather than depend on manual agent judgement, so locking the field prevents accidental mismatches between Impact and Urgency.

Activity 2.1: Configure the Urgency Field

- **Step 1 — Open the UI Policy:** Open the High Impact Control UI Policy created in Milestone 1.
- **Step 2 — Add the action:** Scroll to the UI Policy Actions related list and click New.
- **Step 3 — Choose the field:** Set Field name to Urgency.
- **Step 4 — Set Read only:** Set Read only to True so the field becomes read-only whenever Impact is High. Leave Visible and Mandatory unchanged so the field stays visible.
- **Step 5 — Submit:** Click Submit to save the UI Policy Action.



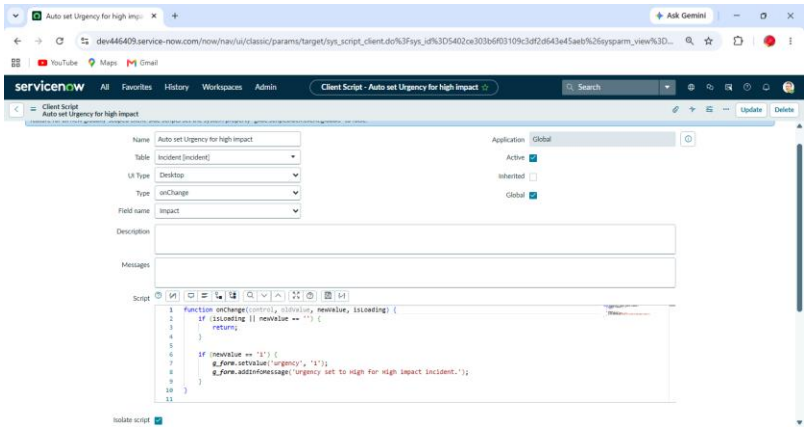
Field	Value
Field name	Urgency
Read-only	True
Visible	Leave alone (default)
Linked UI Policy	High Impact Control

Milestone 3: Create onChange Client Script

Automating the Urgency update the moment Impact changes to High removes an extra manual step for the agent and guarantees consistency as soon as the record is edited, before it is even saved.

Activity 3.1: Create the Client Script

- **Step 1 — Open Client Scripts:** Navigate to System UI → Client Scripts and click New.
- **Step 2 — Enter the details:** Name: Auto set urgency for high impact. Table: Incident. Type: onChange. Field name: Impact. Active: true.
- **Step 3 — Add the script:** Paste the script from Activity 3.2 into the script editor and click Submit.



Field	Value
Name	Auto set urgency for high impact
Table	Incident
Type	onChange
Field name	Impact
Active	true

Activity 3.2: Script

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

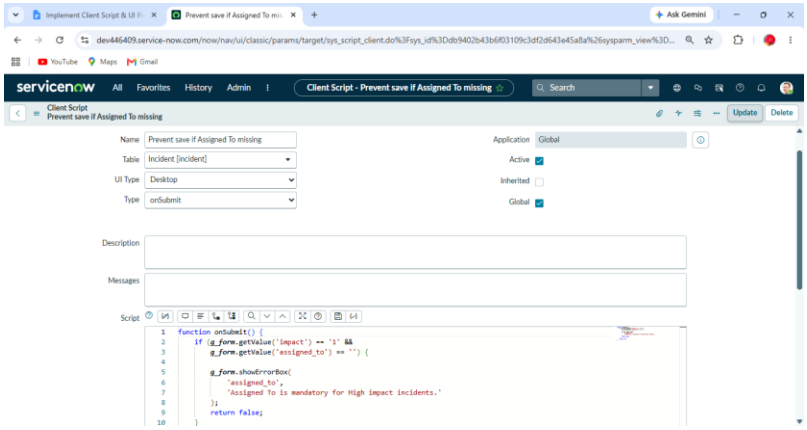
- **isLoading check:** Stops the script from running when the form first loads, so no unwanted message appears.
- **setValue:** Sets Urgency to 1 (High) when Impact changes to 1 (High).
- **addInfoMessage:** Shows a non-blocking message that tells the agent Urgency was set automatically.

Milestone 4: Create onSubmit Client Script (Save Validation)

A UI Policy alone cannot always stop a save outright, so an onSubmit script provides a final, save-time gate that guarantees a High Impact incident cannot be saved without an assigned agent.

Activity 4.1: Create the Client Script

- **Step 1 — Open Client Scripts:** Navigate to System UI → Client Scripts and click New.
- **Step 2 — Enter the details:** Name: Prevent save if Assigned To missing. Table: Incident. Type: onSubmit. Active: true.
- **Step 3 — Add the script:** Paste the script from Activity 4.2 and click Submit.



Field	Value
Name	Prevent save if Assigned To missing
Table	Incident
Type	onSubmit
Active	true

Activity 4.2: Script

```
function onSubmit() {
   if (g_form.getValue('impact') == '1' &&
      g_form.getValue('assigned_to') == '') {

      g_form.showErrorBox(
         'assigned_to',
         'Assigned To is mandatory for High impact incidents.'
      );
      return false;
   }
   return true;
}
```

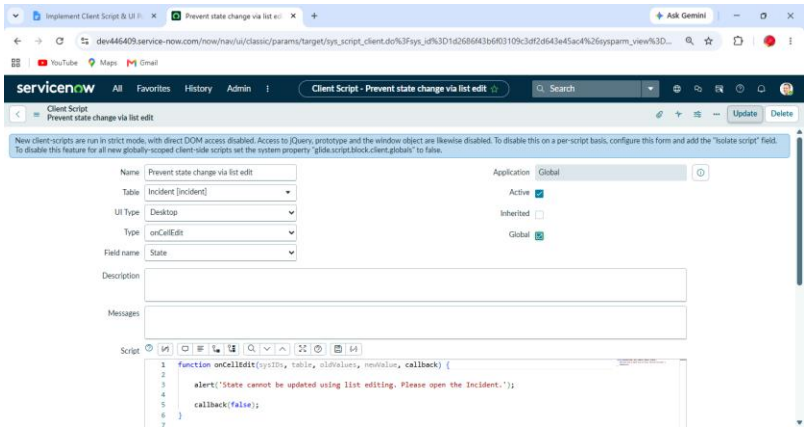
- **getValue:** Reads the current values of Impact and Assigned to from the form.
- **showErrorBox:** Displays the validation message against the Assigned to field and highlights it.
- **return false:** Stops the save. Returning nothing (undefined) would allow ServiceNow to continue saving.

Milestone 5: Create onCellEdit Client Script

List-view inline editing bypasses form-level UI Policies and Client Scripts, so a dedicated onCellEdit script is needed to make sure State changes still go through the full form review process.

Activity 5.1: Create the Client Script

- **Step 1 — Open Client Scripts:** Navigate to System UI → Client Scripts and click New.
- **Step 2 — Enter the details:** Name: Prevent state change via list edit. Table: Incident. Type: onCellEdit. Field name: State. Active: true.
- **Step 3 — Add the script:** Paste the script from Activity 5.2 and click Submit.



Field	Value
Name	Prevent state change via list edit
Table	Incident
Type	onCellEdit
Field name	State
Active	true

Activity 5.2: Script

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');

    callback(false);
}
```

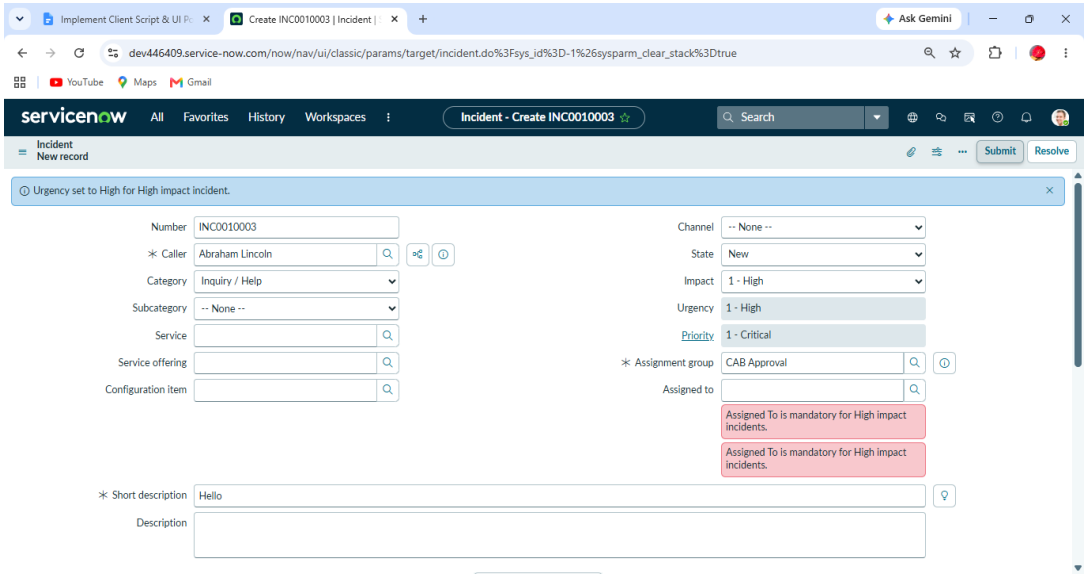
Important Notes:

- Reverse if false must be checked on the High Impact Control UI Policy, otherwise field states may persist incorrectly after Impact is changed away from High.
- onChange scripts should keep the isLoading check to avoid unwanted messages when the form opens.
- onSubmit must explicitly return false when validation fails, and onCellEdit must call callback(false) to cancel the list edit.
- Script field references (impact, urgency, assigned_to, state) must match the backend field names on the Incident table.
- **Milestone 6: Testing and Validation**

Each activity below verifies a specific rule configured across Milestones 1–5, covering mandatory enforcement, successful save, reverse-condition behaviour, and list-edit blocking.

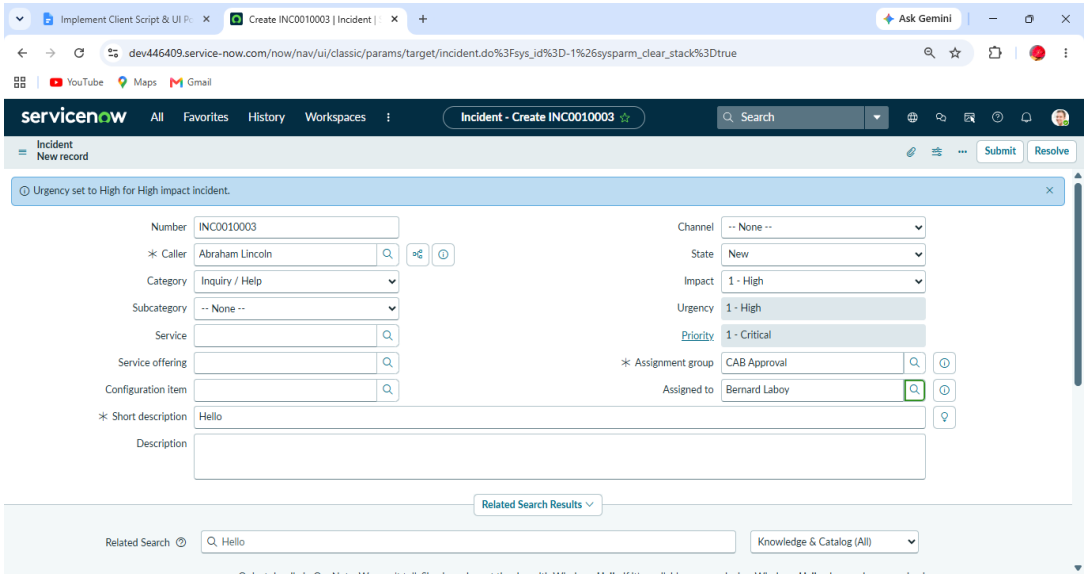
Activity 6.1: Test Mandatory Enforcement

- Go to Incident → Create New to open a new Incident form.
- Select High for the Impact field, which triggers the UI Policy and Client Script behaviours.
- Leave Assigned To empty and click Submit.
- **Expected Result:** The Incident is not saved; an error message states that Assigned To is mandatory; Assignment Group is mandatory and Urgency is read-only.



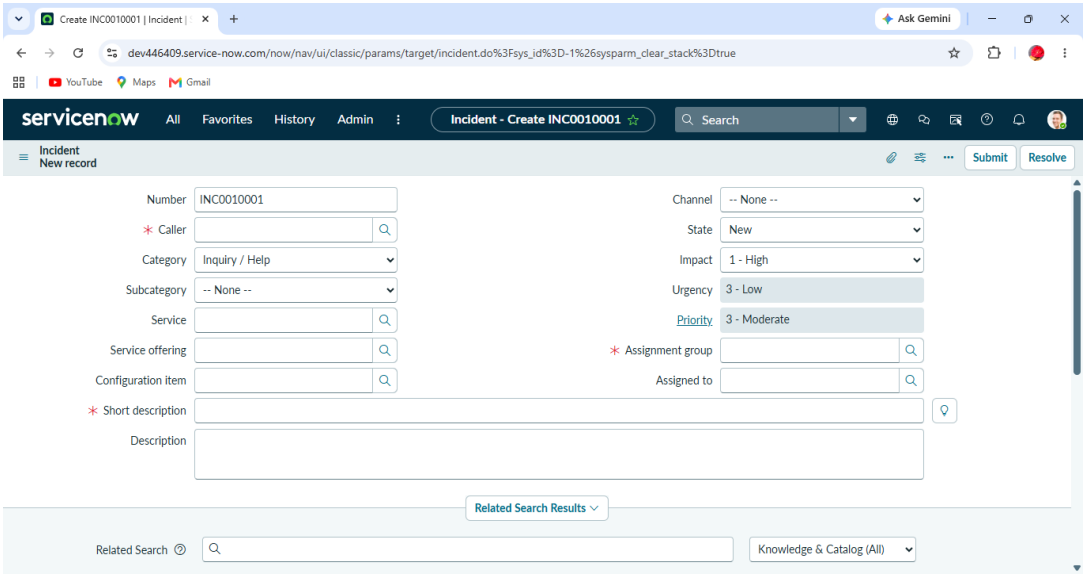
Activity 6.2: Test Successful Save

- On the same form, select a user in the Assigned To field.
- Click Submit.
- **Expected Result:** The Incident saves successfully; Urgency was auto-set to High by the onChange script and remains read-only; Assignment Group was mandatory and completed.



Activity 6.3: Reverse Condition Test

- Open an existing Incident where Impact is currently High.
- Change Impact from High to Medium.
- **Expected Result:** The UI Policy Actions reverse, so Urgency becomes editable again and Assignment Group is no longer mandatory; the record saves without errors.



Activity 6.4: Test List Edit Blocking

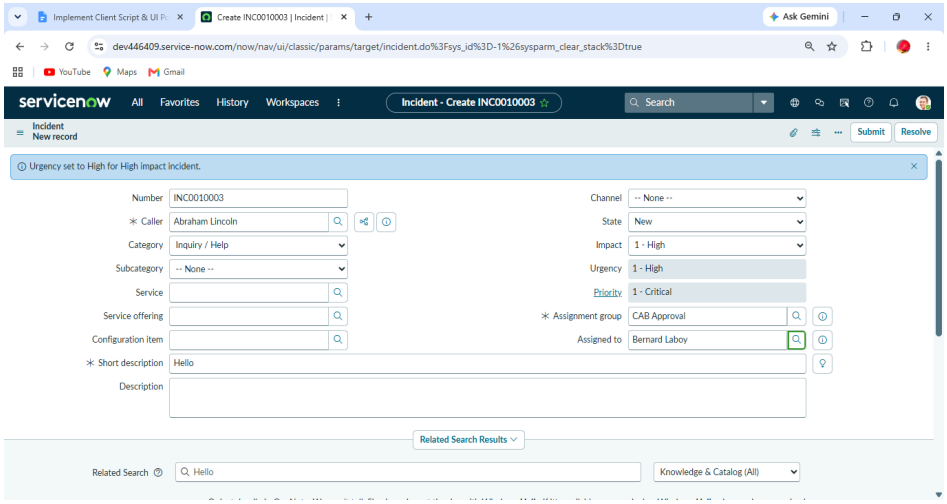
- Go to Incident → All to view the Incident list.
- Double-click the State field on any Incident to edit it directly from the list.
- **Expected Result:** An alert appears stating that State cannot be updated using list editing; the State value remains unchanged.

Test Summary

Test	Feature Tested	Expected Result
Activity 6.1	onSubmit validation / UI Policy	Save blocked; Assigned To error shown.
Activity 6.2	onChange automation / UI Policy	Save succeeds; Urgency auto-set and read-only.
Activity 6.3	UI Policy reverse condition	Mandatory / read-only rules reverse correctly.
Activity 6.4	onCellEdit Client Script	List-view State edit blocked with alert.

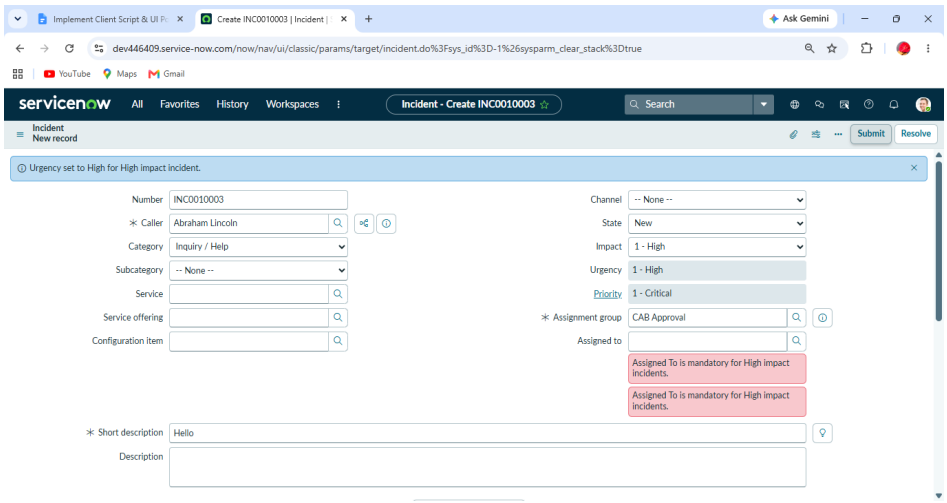
Exploring the Incident Form and List View:

Incident Form – High Impact View:



Description: When Impact is set to High on a new Incident, the form shows the information message "Urgency set to High for High impact incident." Urgency is filled with 1 – High and locked as read-only, and Assignment group is marked mandatory by the High Impact Control UI Policy. With Assigned to selected, the record can be submitted.

Incident Form – Validation Error View:



Description:

If Submit is clicked with Impact set to High and Assigned to empty, the onSubmit Client Script stops the save and shows the message "Assigned To is mandatory for High impact incidents" against the Assigned to field. The record is saved only after the missing value is provided.

Incident List View:

Description:

In the Incident list, double-clicking the State cell to edit it inline triggers the onCellEdit Client Script. An alert tells the user that State cannot be updated using list editing and asks them to open the Incident, and the value remains unchanged.

Conclusion:

Implement Client Script and UI Policy demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behaviour, automate updates, and prevent incorrect data submission on ServiceNow Incident forms. The High Impact Control UI Policy makes Assignment Group mandatory and Urgency read-only when Impact is High and reverses automatically otherwise, while the onChange, onSubmit, and onCellEdit scripts set Urgency, block saves without an Assigned To, and stop direct State edits from the list view. All tests produced the expected results, showing that the solution is lightweight, efficient, and easy to implement. Looking ahead, the same pattern can be extended to the Problem and Change tables, supported by notifications for High Impact incidents saved without an assigned agent, Script Includes for shared validation logic, and Automated Test Framework (ATF) tests to regression-test the behaviour after platform upgrades.